



AskMyDocs

RAG Document Q&A

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the RAG pipeline is wired.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

```
AskMyDocs/
├─ prisma/
│  ├─ schema.prisma      # User, Document, Chunk(embedding), Query
│  ├─ seed.ts            # demo user + ingests sample handbook
│  └─ sample-handbook.md # the demo document
├─ src/
│  ├─ app/
│  │  ├─ page.tsx        # landing
│  │  ├─ chat/           # the workspace (auth-gated)
│  │  └─ api/            # register, auth, documents, ask
│  ├─ lib/               # parse, chunk, embed, retrieve, answer
│  ├─ components/        # Workspace, DocumentLibrary, ChatPanel
│  └─ types/
├─ next.config.ts       # serverExternalPackages for native AI libs
└─ .env                 # DB url, secrets, Gemini key
```

The one rule: AI is touched only in two places — `src/lib/embeddings.ts` (local embeddings) and `src/app/api/ask/route.ts` (remote generation). Everything else goes through Prisma via the singleton client.

1 • Database & lib (the pipeline)

DATABASE BLUEPRINT

prisma/schema.prisma

PostgreSQL datasource + models: `User` ; `Document` (title, filename, mimeType, status, pageCount, chunkCount); `Chunk` (documentId, idx, page, content Text, `embedding` Json number[]); `Query` (logged Q&A with citations Json + model). Enum `DocStatus{PROCESSING,READY,FAILED}` .

PARSE

src/lib/parse.ts

Extract text — PDFs per-page via pdf-parse; TXT/MD as one page.

CHUNK

src/lib/chunk.ts

Structure-aware chunking (~450 chars, heading/paragraph aware, 80-char overlap, page preserved).

★ EMBEDDINGS (LOCAL)

src/lib/embeddings.ts

all-MiniLM-L6-v2 via transformers.js; mean-pool + L2-normalize; batch(32) + single; pipeline cached on `globalThis` .

INGEST ORCHESTRATOR

src/lib/ingest.ts

parse → chunk → embed → `createMany` → set READY/FAILED with counts.

RETRIEVE

src/lib/retrieve.ts + similarity.ts

Embed the query, cosine-score the user's READY chunks, return top-k (default 5).

ANSWER HELPERS

src/lib/answer.ts + constants.ts

Model names, `Citation` type, `toCitations` , `buildContext` , `SYSTEM_PROMPT` , `userPrompt` , `extractiveAnswer` ; the shared `ANSWER_DELIMITER` .

AUTH • SESSION • DB

src/lib/{auth,session,prisma}.ts

NextAuth options; `getUserId()` server helper; singleton Prisma client.

2 · Routes, pages & components

Pages (src/app/)

FILE	PURPOSE
<code>layout.tsx</code>	Root layout, fonts, Providers (SessionProvider).
<code>page.tsx</code>	Landing; redirects to <code>/chat</code> if signed in.
<code>login/page.tsx</code>	Login / register form (demo creds pre-filled).
<code>chat/page.tsx</code>	Auth-gated workspace; server-loads the user's documents.

API routes (src/app/api/)

ENDPOINT	JOB
<code>register</code> · <code>auth/[...nextauth]</code>	Create account (bcrypt) · NextAuth handler.
<code>documents</code> (GET/POST)	List docs · upload a file (multipart) and run the ingest pipeline.
<code>documents/[id]</code> (GET/DELETE)	Fetch one (ownership-checked) · delete (chunks cascade).
★ <code>ask</code> (POST)	Retrieve top-5 + stream a grounded answer with citations; log the Query.

Components (all client)

FILE	PURPOSE
<code>Providers</code> · <code>Navbar</code>	SessionProvider wrapper · top bar + sign-out.
<code>Workspace.tsx</code>	Holds docs list + selected scope; wires Library + ChatPanel.
<code>DocumentLibrary.tsx</code>	Upload UI, doc list + status, scope select, delete.
<code>ChatPanel.tsx</code>	Chat UI: parses the streamed wire format, renders [n] badges + the Sources block + suggestions.

Native-package note: `next.config.ts` marks `@xenova/transformers`, `onnxruntime-node`, `pdf-parse`, `pdfjs-dist`, `sharp` as `serverExternalPackages` so their native binaries aren't bundled.

3 · Commands

COMMAND	WHAT IT DOES
<code>npm install</code>	Install deps (+ <code>prisma generate</code>).
<code>npx prisma db push</code>	Create/sync the PostgreSQL tables.

```
npm run db:seed
```

Seed demo user + ingest the sample handbook (downloads the embedding model on first run).

```
npm run dev
```

Dev server → `localhost:3000` · demo `demo@askmydocs.app / demo1234` .

Gotchas: schema/ `.env` are PostgreSQL though README says MySQL 9 and the committed migration is MySQL-dialect (use `prisma db push`); there is no `.env.example` ; the active AI path is Gemini (Anthropic key empty).